



Stair Rail Chair
Documentation

It has a total of **7 Limit switches** to Control. Here are two types of Limit switches. **NO** (Normally Open) & **NC** (Normally Closed). And a **Motor** to run the chair, left or right.

5 NC Limit Switch
2 NO Limit Switch

5 NC Limit Switch

4 are for braking:

- A. Left Stop.
- B. Left Edge.
- C. Right Stop.
- D. Right Edge.

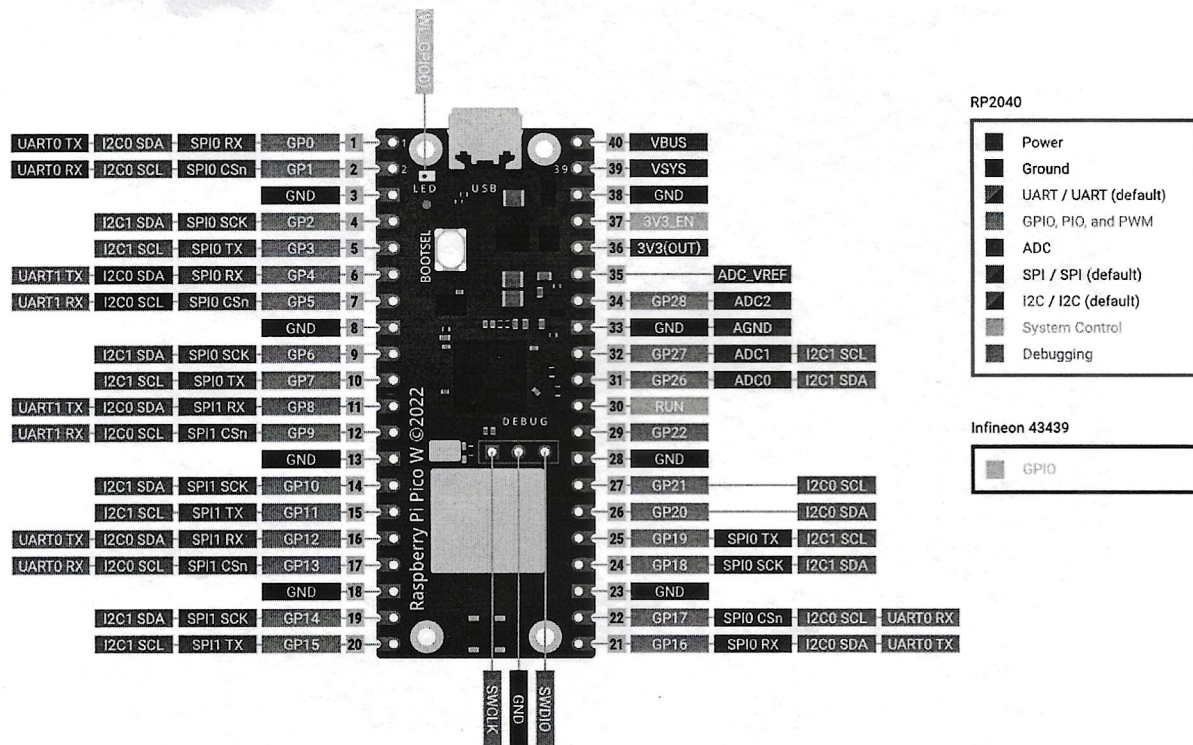
1 is for Safety Lock.

2 NO Limit Switch

To control the motor rotation for left/right.

- A. UP (RPWM).
- B. DOWN (LPWM).

We are using a Raspberry Pi Pico W to operate it.



Here is the connection of the Switches, motors, relays to the Raspberry Pi Pico W

Safety Limit Switches

Left Edge	GPIO 6
Left Stop	GPIO 8
Right Stop	GPIO 9
Safety Circuit	GPIO 3
Right Edge	GPIO 7

Control Up & Down Limit Switches

Call Up	GPIO 20
	RPWM
Call Down	GPIO 21
	LPWM

Output

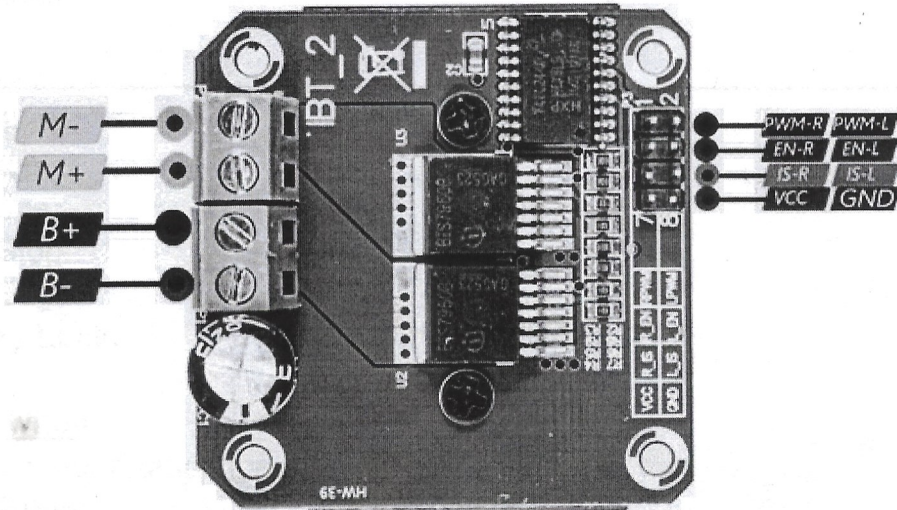
Buzzer	GPIO 18	Motor Control	RPWM	GPIO 26
NeoPixel RGB	GPIO 28		LPWM	GPIO 27

Brk
relay GPIO 1

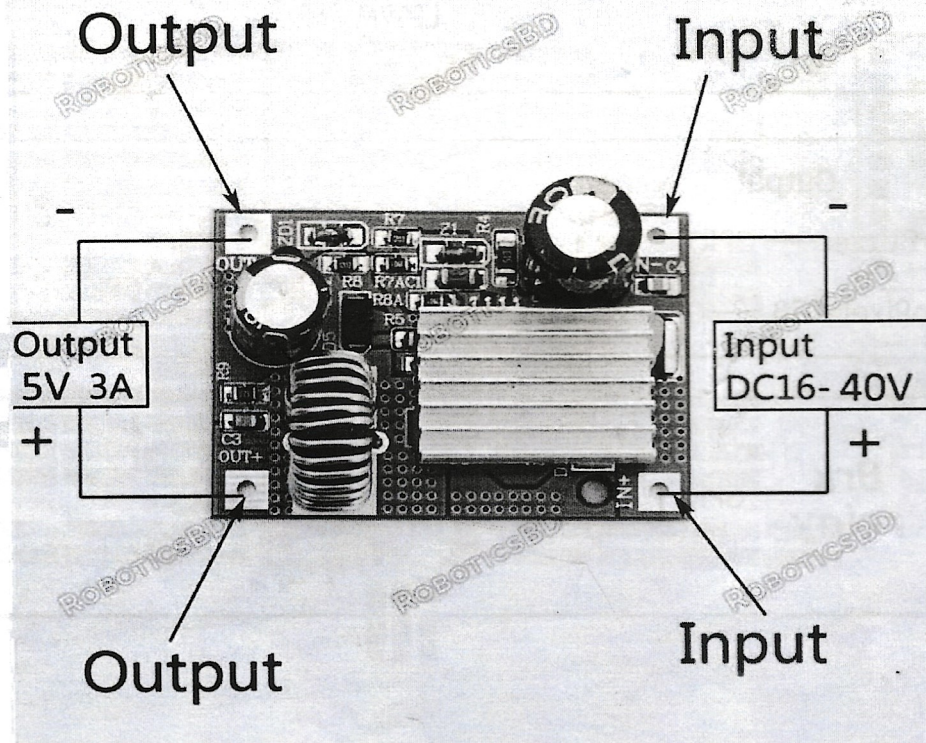
Crg
relay GPIO 2

Here are the other components for power.

1. Adapter 24~27V
2. Battery 12V+12V =24V (Series Connection)
3. BTS7960 Motor Driver



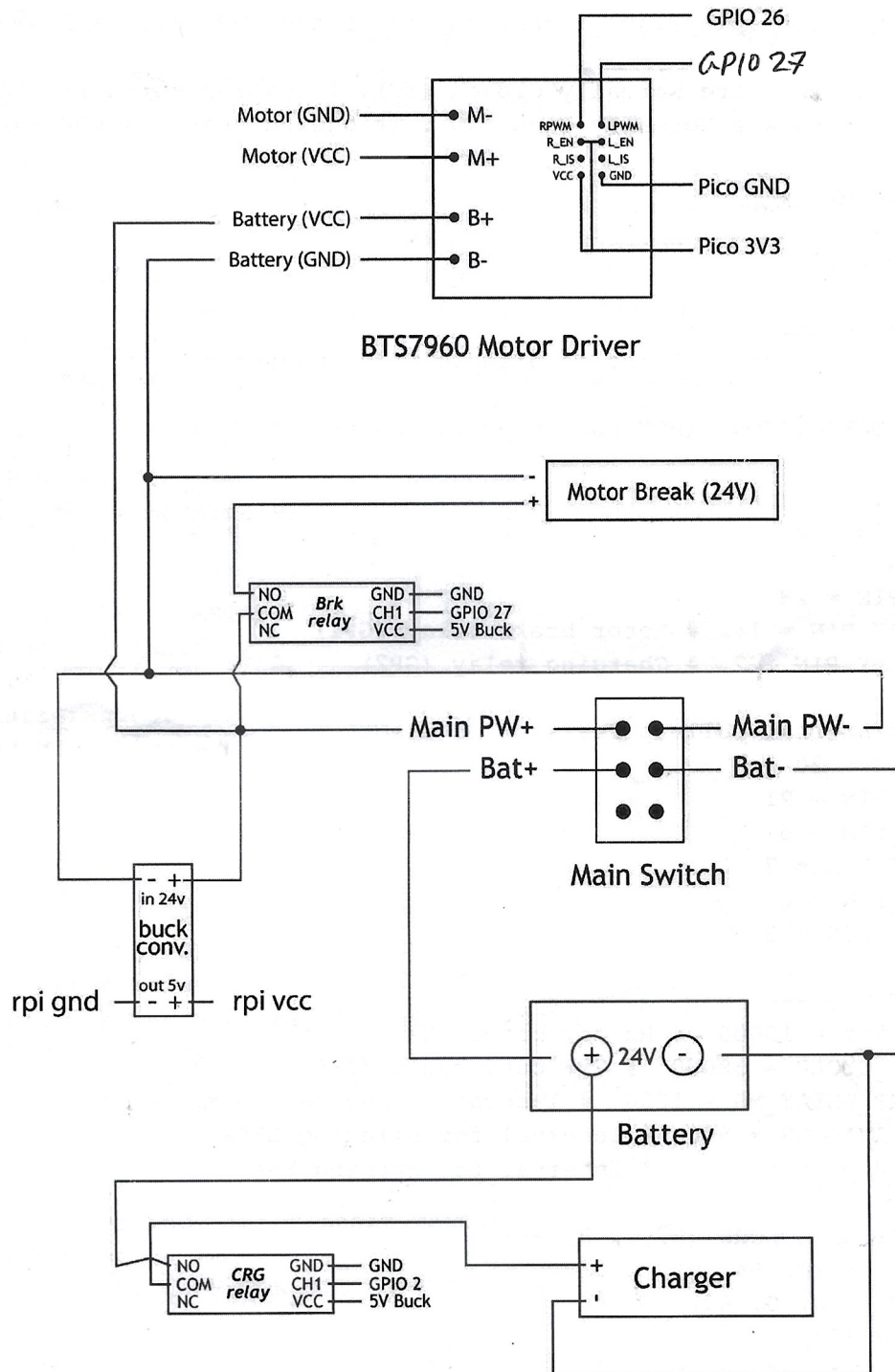
4. Step Down Buck Converter (16~40V to 5V 3A)



5. Power Bank to supply a constant power to the Raspberry Pi.

Here is the power connection of this total system.

Power Connection



Source Code

```
# main.py - MicroPython Stairlift Controller
# VERSION: 1-second delay and direction-specific immediate stop.
# CONFIRMED LOGIC:
# - Left Station (pin_left_stop/edge) is at the BOTTOM. Allows UP movement
  only.
# - Right Station (pin_right_stop/edge) is at the TOP. Allows DOWN movement
  only.
# - Limit switches are Normally Closed (NC). Triggered state is HIGH (1).
# - Call buttons are Normally Open (NO). Triggered state is LOW (0).

import machine
import time
import neopixel

# --- CONFIGURATION ---
DEBUG = True # Set to False to disable print messages

# --- PIN DEFINITIONS (BCM numbering for Raspberry Pi Pico) ---
# Outputs
PWM_UP_PIN = 26
PWM_DOWN_PIN = 27
BUZZER_PIN = 18
NEOPIXEL_PIN = 28
BRAKE_RELAY_PIN = 1 # Motor brake relay (GP1)
CHARGE_RELAY_PIN = 2 # Charging relay (GP2)

# Inputs (Internal pull-up resistor is used)
CALL_UP_PIN = 20
CALL_DOWN_PIN = 21
LEFT_EDGE_PIN = 6
RIGHT_EDGE_PIN = 7
LEFT_STOP_PIN = 8
RIGHT_STOP_PIN = 9

# --- SETTINGS ---
PWM_FREQUENCY = 15000 # Hz for motor PWM
MOTOR_DUTY_CYCLE = 38000 # 75% duty cycle (0-65535)
MOTOR_START_DELAY_MS = 1000 # 1-second delay before motor starts
BLINK_INTERVAL_MS = 500 # Interval for blinking LEDs
BEEP_INTERVAL_MS = 750 # Interval for warning beeps

# --- NEOPIXEL COLORS (R, G, B) ---
COLOR_GREEN = (0, 64, 0)
COLOR_BLUE = (0, 0, 64)
COLOR_RED = (64, 0, 0)
COLOR_PURPLE = (64, 0, 64)
COLOR_ORANGE = (255, 100, 0)
COLOR_BLACK = (0, 0, 0)

# --- HARDWARE INITIALIZATION ---
# Outputs
```

```

pwm_up = machine.PWM(machine.Pin(PWM_UP_PIN))
pwm_down = machine.PWM(machine.Pin(PWM_DOWN_PIN))
pwm_up.freq(PWM_FREQUENCY)
pwm_down.freq(PWM_FREQUENCY)
pwm_up.duty_u16(0)
pwm_down.duty_u16(0)

buzzer = machine.Pin(BUZZER_PIN, machine.Pin.OUT)
brake_relay = machine.Pin(BRAKE_RELAY_PIN, machine.Pin.OUT)
charge_relay = machine.Pin(CHARGE_RELAY_PIN, machine.Pin.OUT)
np = neopixel.NeoPixel(machine.Pin(NEOPIXEL_PIN), 1)

# Inputs
pin_call_up = machine.Pin(CALL_UP_PIN, machine.Pin.IN, machine.Pin.PULL_UP)
pin_call_down = machine.Pin(CALL_DOWN_PIN, machine.Pin.IN,
machine.Pin.PULL_UP)
pin_left_edge = machine.Pin(LEFT_EDGE_PIN, machine.Pin.IN,
machine.Pin.PULL_UP)
pin_right_edge = machine.Pin(RIGHT_EDGE_PIN, machine.Pin.IN,
machine.Pin.PULL_UP)
pin_left_stop = machine.Pin(LEFT_STOP_PIN, machine.Pin.IN,
machine.Pin.PULL_UP)
pin_right_stop = machine.Pin(RIGHT_STOP_PIN, machine.Pin.IN,
machine.Pin.PULL_UP)

# --- GLOBAL STATE VARIABLES ---
system_state = "INIT"
last_state = ""
motor_start_request_time = 0
requested_direction = None
is_moving = False
led_is_on = False
last_blink_time = 0
last_beep_time = 0

# --- HELPER FUNCTIONS ---
def debug_print(message):
    """Prints a message only if DEBUG is True."""
    if DEBUG:
        print(f"[{time.ticks_ms()}] {message}")

def motor_stop():
    """Immediately stops all motor activity and resets movement state."""
    global is_moving, motor_start_request_time, requested_direction
    if is_moving or requested_direction:
        debug_print("MOTOR STOP COMMANDED")

    pwm_up.duty_u16(0)
    pwm_down.duty_u16(0)
    brake_relay.low() # Engage the brake
    is_moving = False
    motor_start_request_time = 0
    requested_direction = None

```

```

def beep(duration_ms=50):
    """Generates a short beep."""
    buzzer.high()
    time.sleep_ms(duration_ms)
    buzzer.low()

# --- CORE LOGIC ---
def evaluate_system_state(inputs):
    """Determines the current state of the system based on sensor
    inputs."""
    # Check for ZONED states
    if inputs['left_stop'] or inputs['left_edge']:
        return "ZONED_BOTTOM"

    if inputs['right_stop'] or inputs['right_edge']:
        return "ZONED_TOP"

    # If not zoned, it's in a normal state
    return "HEALTHY"

def handle_indicators(current_state):
    """Manages the Neopixel and Buzzer based on the system state
    (non-blocking)."""
    global led_is_on, last_blink_time, last_beep_time

    color_to_set = COLOR_BLACK
    blink = False
    beep_warning = False

    if is_moving:
        color_to_set = COLOR_BLUE
        blink = True
    elif current_state == "HEALTHY":
        color_to_set = COLOR_GREEN
    elif current_state == "UNHEALTHY": # This state is unreachable but
        # logic is kept for safety
        color_to_set = COLOR_RED
        blink = True
        beep_warning = True
    elif current_state == "ZONED_BOTTOM":
        if charge_relay.value() == 0:
            color_to_set = COLOR_ORANGE
        else:
            color_to_set = COLOR_PURPLE
    elif current_state == "ZONED_TOP":
        if charge_relay.value() == 0:
            color_to_set = COLOR_ORANGE
        else:
            color_to_set = COLOR_PURPLE

    # Blinking logic
    if blink:

```

```

        if time.ticks_diff(time.ticks_ms(), last_blink_time) >
BLINK_INTERVAL_MS:
            last_blink_time = time.ticks_ms()
            led_is_on = not led_is_on
            np[0] = color_to_set if led_is_on else COLOR_BLACK
            np.write()
    else:
        np[0] = color_to_set
        np.write()

    # Beeping logic
    if beep_warning:
        if time.ticks_diff(time.ticks_ms(), last_beep_time) >
BEEP_INTERVAL_MS:
            last_beep_time = time.ticks_ms()
            buzzer.high()
            elif time.ticks_diff(time.ticks_ms(), last_beep_time) > 100: # Beep
duration
                buzzer.low()
        else:
            buzzer.low()

# --- MAIN LOOP ---
debug_print("Stairlift Controller Initialized. Starting main loop.")
beep(100) # Initialization beep

while True:
    try:
        # 1. READ ALL INPUTS
        inputs = {
            'call_up': pin_call_up.value() == 0,
            'call_down': pin_call_down.value() == 0,
            'left_edge': pin_left_edge.value() == 1,
            'right_edge': pin_right_edge.value() == 1,
            'left_stop': pin_left_stop.value() == 1,
            'right_stop': pin_right_stop.value() == 1,
        }

        # 2. EVALUATE SYSTEM STATE
        system_state = evaluate_system_state(inputs)
        system_state = evaluate_system_state(inputs)

```

```

        # If moving UP, check for TOP limit switches
        if is_moving_up and (inputs['right_stop'] or
inputs['right_edge']):
            debug_print("IMMEDIATE STOP TRIGGERED: Hit top limit while
moving UP.")
            motor_stop()
            continue

        # If moving DOWN, check for BOTTOM limit switches
        if is_moving_down and (inputs['left_stop'] or
inputs['left_edge']):
            debug_print("IMMEDIATE STOP TRIGGERED: Hit bottom limit
while moving DOWN.")
            motor_stop()
            continue

# 4. HANDLE RELAYS
if system_state == "ZONED_TOP" or system_state == "ZONED_BOTTOM":
    charge_relay.low()
else:
    charge_relay.high()

# 5. HANDLE MOVEMENT LOGIC
if not is_moving and not requested_direction:
    can_move_up = system_state in ["HEALTHY", "ZONED_BOTTOM"]
    can_move_down = system_state in ["HEALTHY", "ZONED_TOP"]

    if inputs['call_up'] and can_move_up:
        debug_print("Call UP button pressed. Starting 1s delay.")
        requested_direction = "UP"
        motor_start_request_time = time.time()
        brake_relay.high()
        beep(50)
    elif inputs['call_down'] and can_move_down:
        debug_print("Call DOWN button pressed. Starting 1s delay.")
        requested_direction = "DOWN"
        motor_start_request_time = time.time()
        brake_relay.high()
        beep(50)

    if requested_direction and time.time() - motor_start_request_time >= MOTOR_START_DELAY_MS:
        is_moving = True
        if requested_direction == "UP":
            debug_print("1s delay finished. Starting motor UP.")
            pwm_down.duty_u16(0)
            pwm_up.duty_u16(MOTOR_DUTY_CYCLE)
        elif requested_direction == "DOWN":
            debug_print("1s delay finished. Starting motor DOWN.")
            pwm_up.duty_u16(0)
            pwm_down.duty_u16(MOTOR_DUTY_CYCLE)

```

```

        requested_direction = None
        motor_start_request_time = 0

    if is_moving:
        if (pwm_up.duty_u16() > 0 and not inputs['call_up']) or \
            (pwm_down.duty_u16() > 0 and not inputs['call_down']):
            debug_print("Call button released. Stopping motor.")
            motor_stop()

    if requested_direction:
        if (requested_direction == "UP" and not inputs['call_up']) or \
            (requested_direction == "DOWN" and not inputs['call_down']):
            debug_print("Call button released during delay. Cancelling
move.")
            motor_stop()

# 6. HANDLE INDICATORS
handle_indicators(system_state)

time.sleep_ms(10)

except Exception as e:
    debug_print(f"An error occurred: {e}")
    motor_stop()
    while True:
        np[0] = COLOR_RED
        np.write()
        time.sleep_ms(100)
        np[0] = COLOR_BLACK
        np.write()
        time.sleep_ms(100)

```